

Phase 1: Foundation & Core Mechanics (Weeks 1–3)

Week 1: Project Initialization & Git Strategy

- **What I will learn:** Professional repository architecture, semantic Git commit messaging.
- **What I will do in GitHub:** * Create a public repository named ClipStack.
 - Add a standard Java .gitignore file to exclude IDE build directories (.idea/, bin/, out/).
 - Initialize a main branch and cut a development branch.
 - Write a baseline README.md containing the project objectives and architecture diagram.
- **What I will submit:** Live GitHub repository URL .

Week 2: Cracking the Java Clipboard API

- **What I will learn:** How Java interacts with native operating system subsystems using java.awt.Toolkit and the nuances of data flavors (DataFlavor.stringFlavor).
- **What I will do in GitHub:** * Create a branch named feature/clipboard-core.
 - Commit a lightweight, standalone Java class that reads the current string content of the OS clipboard and prints it to the console.
 - Merge the branch into development via a Pull Request (PR).
- **What I will submit:** A working snippet or a short test log demonstrating successful text extraction from your system clipboard.

Week 3: Designing the Background Polling Thread

- **What I will learn:** Multi-threading basics, scheduling loops via `ScheduledExecutorService` or `Thread.sleep()`, and handling thread interruptions gracefully.
- **What I will do in GitHub:** * Create a branch named `feature/polling-engine`.
 - Implement a background worker thread that checks the system clipboard every 1000ms.
 - Ensure the thread compares the current text with the previous text to prevent logging duplicate entries consecutively.
- **What I will submit:** Console logs proving the background thread detects changes in real-time as you copy different texts across your operating system.

Phase 2: Memory & Data Management (Weeks 4–6)

Week 4: Implementing the History Stack Logic

- **What I will learn:** Custom encapsulation using Java Collections, specifically managing limits on a `LinkedList` to act as a FIFO (First-In, First-Out) rolling cache.
- **What I will do in GitHub:** * Create a branch named `feature/history-manager`.
 - Build a dedicated `ClipboardHistory` class. Write custom methods to handle insertions: if the list size exceeds 10, drop the oldest snippet automatically.
- **What I will submit:** Unit test outputs or console traces showing the collection dynamically keeping exactly the last 10 copied strings without growing indefinitely.

Week 5: Mid-Project Consolidation & Console UI

- **What I will learn:** Structuring clean interactive text menus, managing system inputs, and formatting multi-line string outputs cleanly.
- **What I will do in GitHub:** * Create a branch named feature/console-interface.
 - Connect the background tracking thread to a live, loop-driven console menu allowing users to view the current list of 10 items and pick a number to re-copy it.
 - Perform a comprehensive merge of all stable features into your development branch.
- **What I will submit:** A mid-project progress documentation report detailing your current codebase state, architectural structure, and any roadblocks hit.

Week 6: Re-Writing to the System Clipboard

- **What I will learn:** Transferring data back to the OS selection layer via StringSelection and updating system flavors programmatically.
- **What I will do in GitHub:** * Create a branch named feature/write-back.
 - Write the execution logic: when a user selects item #3 from your history stack, the application must inject that exact string back into the operating system's clipboard so they can hit Ctrl+V anywhere.
- **What I will submit:** A functional demo run showing text traversing both ways: OS Clipboard to ClipStack Memory to OS Clipboard.

Phase 3: UI Enhancement & Edge Cases (Weeks 7–8)

Week 7: Elevating to the System Tray UI (Optional but Recommended)

- **What I will learn:** Native OS integration using Java's `java.awt.SystemTray` and `java.awt.PopupMenu` to make the application run quietly in the taskbar.
- **What I will do in GitHub:** * Create a branch named `feature/system-tray-ui`.
 - Replace or supplement the console menu with a functional icon in your computer's taskbar tray that displays the history upon clicking.
- **What I will submit:** Screenshots of your application running smoothly within the Windows System Tray or macOS Menu Bar.

Week 8: Bulletproofing Edge Cases & Memory Leaks

- **What I will learn:** Exception handling strategies for data mismatches (e.g., what happens if a user copies an *image* or a *file* instead of text?), and defensive copying.
- **What I will do in GitHub:** * Create a branch named `bugfix/robustness-handling`.
 - Add defensive check clauses (`clipboard.isDataFlavorSupported()`) to entirely ignore non-text copies without crashing the background service.
- **What I will submit:** A simple test report detailing your application's behavior when encountering rich text, files, images, and null data streams.

Phase 4: Refactoring, Testing & Launch (Weeks 9–10)

Week 9: Optimization & Final Code Review

- **What I will learn:** Code refactoring techniques, ensuring proper separation of concerns (UI logic separated from Clipboard management logic), and code optimization.
- **What I will do in GitHub:** * Perform code cleanups directly on the development branch.
 - Open the final, comprehensive Pull Request from development into main.
 - Write an extensive PR description detailing the complete system flow.
- **What I will submit:** Cleaned code repository with closed PRs and a finalized architecture validation check.

Week 10: Release & Final Presentation Packaging

- **What I will learn:** Compiling executable artifacts (.jar distribution files) and documenting production-ready technical source material.
- **What I will do in GitHub:** * Merge the final codebase into main.
 - Draft a highly polished README.md complete with instructions on installation, user guides, and execution commands.
 - Use GitHub Releases to tag version v1.0.0 and attach your compiled runnable .jar file.
- **What I will submit:** A functional, compiled project executable (.jar), the complete GitHub source link, and your project presentation/documentation.